



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

具身智能大作业报告

学 院：未来精工技术学院

专 业：卓越班

班 级：特立 2503

学 号：1120250799

姓 名：卢振越

任课教师：岳裕丰

二〇二六年二月二十六日

目录

1. DQN 算法原理 3

1.1 问题背景与算法目标 3

1.2 核心思想 3

1.3 算法关键机制 3

1.3.1 ϵ -greedy 动作选择 3

1.3.2 Experience Replay 3

1.3.3 目标函数与 TD 误差 4

1.3.4 网络结构与训练设置 4

1.4 算法流程 5

2. A2C 算法原理 6

2.1 算法目标与核心思想 6

2.2 算法关键机制 6

2.2.1 网络结构与动作采样 6

2.2.2 n-step 回报与优势估计 6

2.2.3 优化目标 7

2.3 算法流程 8

3. 实现过程 8

3.1 运行环境与依赖配置 8

3.2 模型构建 8

3.3 训练主循环实现 9

3.4 训练过程可视化与平滑处理 9

3.5 复现实验配置 9

3.6 训练执行、调试与结果保存 9

4. 算法比较及原因分析 9

4.1 训练结果对比 9

4.2	训练结果分析	10
4.3	训练耗时分析	11
5.	实验总结	11

1. DQN 算法原理

1.1 问题背景与算法目标

传统 Q-learning 依赖表格存储动作价值函数，难以直接处理高维像素输入。DQN (Deep Q-Network) 通过深度卷积神经网络近似动作价值函数 $Q(s, a)$ ，目标是在 Atari 等离散动作环境中，从原始图像输入端到端学习控制策略。

1.2 核心思想

DQN 将 Q-learning 与卷积神经网络结合：输入为状态 s ，输出为该状态下各离散动作的价值估计。策略由

$$\pi(s) = \arg \max_a Q(s, a; \theta)$$

给出，并通过时序差分目标进行迭代更新。

1.3 算法关键机制

1.3.1 ϵ -greedy 动作选择

训练阶段采用 ϵ -greedy 策略：以概率 ϵ 随机选动作，以概率 $1 - \epsilon$ 选择当前估计价值最大的动作，这样可以让智能体充分探索各种动作，有效防止陷入局部最优解。此外，为平衡探索与学习的策略， ϵ 随训练过程逐步减小。也就是说，学习的策略随着训练的过程越来越重要。

1.3.2 Experience Replay

智能体与环境交互得到的转移样本会被存入回放缓冲区。训练时，从缓冲区随机采样小批量数据来更新参数，以削弱样本时序相关性并提升数据利用率。

1.3.3 目标函数与 TD 误差

给定转移样本 (s, a, r, s') ，DQN 的目标值定义为

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-),$$

损失函数为

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2].$$

其中， θ 为在线网络参数， θ^- 为目标网络参数。目标网络参数 θ^- 由在线网络参数 θ 周期性复制得到。

1.3.4 网络结构与训练设置

按照论文设定，输入状态由最近 4 帧预处理图像堆叠构成，尺寸为 $4 \times 84 \times 84$ 。图像预处理流程包括：原始 Atari 帧转灰度、缩放至 84×84 、按时间顺序堆叠 4 帧。网络结构如下：

- Conv1: 32 个卷积核，大小 8×8 ，步长 4，ReLU；
- Conv2: 64 个卷积核，大小 4×4 ，步长 2，ReLU；
- Conv3: 64 个卷积核，大小 3×3 ，步长 1，ReLU；
- 全连接层：512 个隐藏单元，ReLU；
- 输出层：维度为动作空间大小 $|\mathcal{A}|$ 。

优化器使用 RMSProp，折扣因子取 $\gamma = 0.99$ ，并结合经验回放与目标网络进行稳定训练。

1.4 算法流程

Algorithm 1 Deep Q-Learning with Experience Replay

```

1: 初始化回放缓冲区  $D$ ，容量为  $N$ 
2: 初始化在线网络  $Q(\cdot, \cdot; \theta)$ 
3: 初始化目标网络  $Q(\cdot, \cdot; \theta^-)$ ，并令  $\theta^- \leftarrow \theta$ 
4: for episode = 1, 2, ...,  $M$  do
5:   重置环境并获得初始状态，构造预处理状态  $\phi_1$ 
6:   for  $t = 1, 2, \dots, T$  do
7:     以概率  $\epsilon$  随机选择动作  $a_t$ 
8:     否则选择  $a_t \leftarrow \arg \max_a Q(\phi_t, a; \theta)$ 
9:     执行动作  $a_t$ ，观察奖励  $r_t$  和下一帧，并构造  $\phi_{t+1}$ 
10:    将转移  $(\phi_t, a_t, r_t, \phi_{t+1}, \text{done})$  存入  $D$ 
11:    if 达到训练频率 then
12:      从  $D$  随机采样小批量转移  $(\phi_j, a_j, r_j, \phi_{j+1}, \text{done}_j)$ 
13:      计算目标值
14:       $y_j \leftarrow r_j$ ，若  $\text{done}_j = \text{True}$ 
15:       $y_j \leftarrow r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta^-)$ ，若  $\text{done}_j = \text{False}$ 
16:      对损失  $(y_j - Q(\phi_j, a_j; \theta))^2$  做梯度下降，更新  $\theta$ 
17:    end if
18:    if 达到目标网络同步频率 then
19:       $\theta^- \leftarrow \theta$ 
20:    end if
21:  end for
22: end for

```

2. A2C 算法原理

在本次实验中选择 A2C，主要有两点考虑：第一，A2C 支持并行环境采样，单位时间内可以收集更多交互数据，训练时间成本低；第二，本实验已实现 DQN，引入 A2C 可以形成“值函数方法 vs 策略梯度方法”的对比，有助于更全面地分析不同算法在 Atari 任务中的表现。

2.1 算法目标与核心思想

A2C (Advantage Actor-Critic) 通过“策略网络 (Actor) + 价值网络 (Critic)”联合优化离散的动作策略。Actor 输出动作概率分布，Critic 估计状态价值，用于构造优势函数以降低策略梯度方差。与单轨迹更新相比，本次实现采用并行环境与固定步长采样，提高数据吞吐与训练稳定性。

2.2 算法关键机制

2.2.1 网络结构与动作采样

实现过程中，使用共享卷积特征提取器，输入为 $4 \times 84 \times 84$ 图像堆叠。共享主干后接两个分支：

- Actor 头：输出各动作 logits，经 softmax 得到策略 $\pi_{\theta}(a | s)$ ；
- Critic 头：输出标量状态价值 $V_{\theta}(s)$ 。

在每个环境步，按分类分布 $\pi_{\theta}(\cdot | s)$ 采样动作并与环境交互。

2.2.2 n-step 回报与优势估计

定义 R_t 为从时刻 t 出发的 n-step 目标回报，即未来奖励的折扣累积，并在末端使用 bootstrap 值进行补全； $V_{\theta}(s_t)$ 表示 Critic 对状态 s_t 的价值估计，即在当前策略下从该状态出发的期望折扣回报。优势 A_t 用于衡量“当前动作结果相对状态平均水平的好坏”，从而降低策略梯度更新的方差。设并行环境数为 N ，每次更新前滚动采样 n

步（实现中 $n = 5$ ）。对每个时间步按反向递推计算 **n-step** 回报：

$$R_t = r_t + \gamma m_t R_{t+1},$$

其中 $m_t \in \{0, 1\}$ 为终止掩码；最后一步使用 **bootstrap** 值 $V(s_{t+n})$ 初始化。优势定义为

$$A_t = R_t - V_\theta(s_t),$$

并在实现中做标准化以稳定训练：

$$\hat{A}_t = \frac{A_t - \mu_A}{\sigma_A + 10^{-8}}.$$

2.2.3 优化目标

策略损失、价值损失与熵正则项分别为

$$L_\pi = -\mathbb{E}[\log \pi_\theta(a_t | s_t) \hat{A}_t],$$

$$L_V = \mathbb{E}[(R_t - V_\theta(s_t))^2],$$

$$L_H = \mathbb{E}[\mathcal{H}(\pi_\theta(\cdot | s_t))].$$

总损失为

$$L = L_\pi + 0.5L_V - 0.01L_H.$$

实现中使用 **RMSProp** ($lr = 7 \times 10^{-4}$, $\alpha = 0.99$, $\epsilon = 10^{-5}$), 并进行梯度裁剪 ($\|g\|_2 \leq 0.5$)。

2.3 算法流程

Algorithm 2 A2C with Parallel Atari Environments

- 1: 初始化并行 Atari 环境、A2C 网络与优化器
 - 2: 重置环境，得到初始观测
 - 3: **while** 未达到总步数上限 **do**
 - 4: 在 N 个环境上连续采样 n 步：记录 $\log \pi(a_t | s_t)$ 、 $V(s_t)$ 、 r_t 、mask、熵
 - 5: 用最后观测的价值估计作为 bootstrap，反向计算 n-step 回报 R_t
 - 6: 计算优势 $A_t = R_t - V(s_t)$ 并标准化
 - 7: 计算总损失 $L = L_\pi + 0.5L_V - 0.01L_H$
 - 8: 反向传播并更新参数，执行梯度裁剪
 - 9: 周期性记录奖励曲线与损失曲线，并按步数保存模型参数
 - 10: **end while**
-

3. 实现过程

3.1 运行环境与依赖配置

为保证实验可复现性与依赖隔离，本项目首先使用 Python 的 `venv` 创建独立虚拟环境。在该环境中安装与 CUDA 12.8 对应的 PyTorch，用于神经网络训练与 GPU 加速；安装 ALE (Arcade Learning Environment) 用于 Atari 游戏环境交互；安装 OpenCV (cv2) 用于训练后策略效果回放与结果展示。

3.2 模型构建

基于 PyTorch 分别实现 DQN 与 A2C 两类网络。两类模型均以预处理后的 Atari 图像堆叠作为输入。DQN 输出各离散动作的 Q 值，用于值函数学习；A2C 采用共享特征提取主干，并分为 Actor 与 Critic 两个分支，分别输出动作策略分布与状态价值估计。

3.3 训练主循环实现

训练主循环的流程统一为“动作选择—环境交互—数据记录—损失计算—反向传播—参数更新”。其中，DQN 包含经验回放采样与目标网络同步；A2C 包括计算 n-step 回报。该循环实现了从数据采集到参数优化的闭环训练。

3.4 训练过程可视化与平滑处理

为便于观察训练趋势，实验中实时绘制 reward 随训练进程变化曲线。考虑到强化学习奖励存在较大随机波动，在原始曲线基础上增加滑动平均平滑曲线，用于更稳定地反映策略性能变化。

3.5 复现实验配置

为保证与算法原理部分一致，按照论文配置优化器与关键超参数，包括折扣因子、学习率、batch size、并行环境数、n-step 长度、目标网络更新频率等。

3.6 训练执行、调试与结果保存

在正式训练阶段，逐步完成运行调试，重点排查张量维度问题。训练稳定后，按一定周期保存模型参数，用于展示训练成果。

4. 算法比较及原因分析

4.1 训练结果对比

本实验分别统计了 A2C 与 DQN 的 reward 变化曲线，并使用滑动平均进行平滑处理，结果如图 4-1和图 4-2所示。

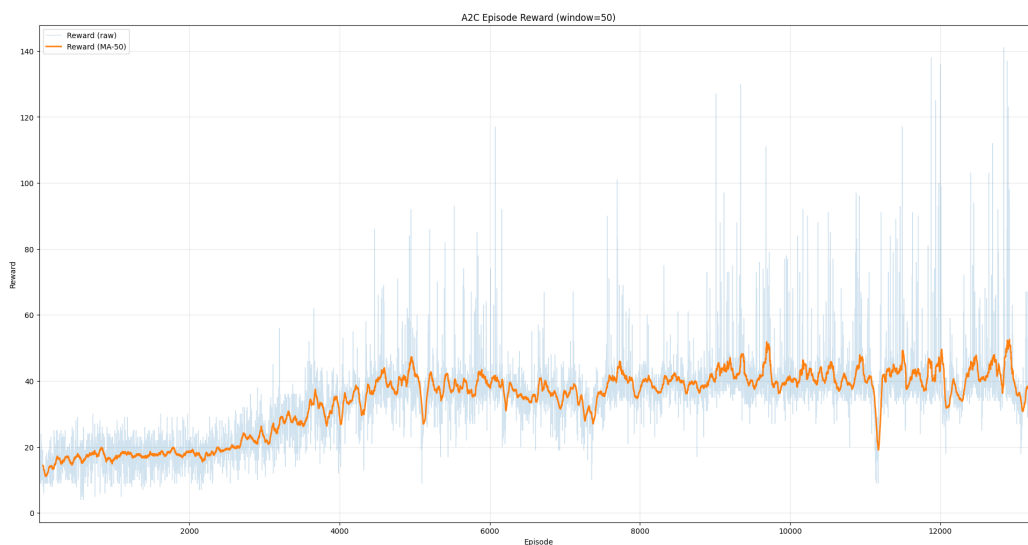


图 4-1 A2C 训练奖励曲线

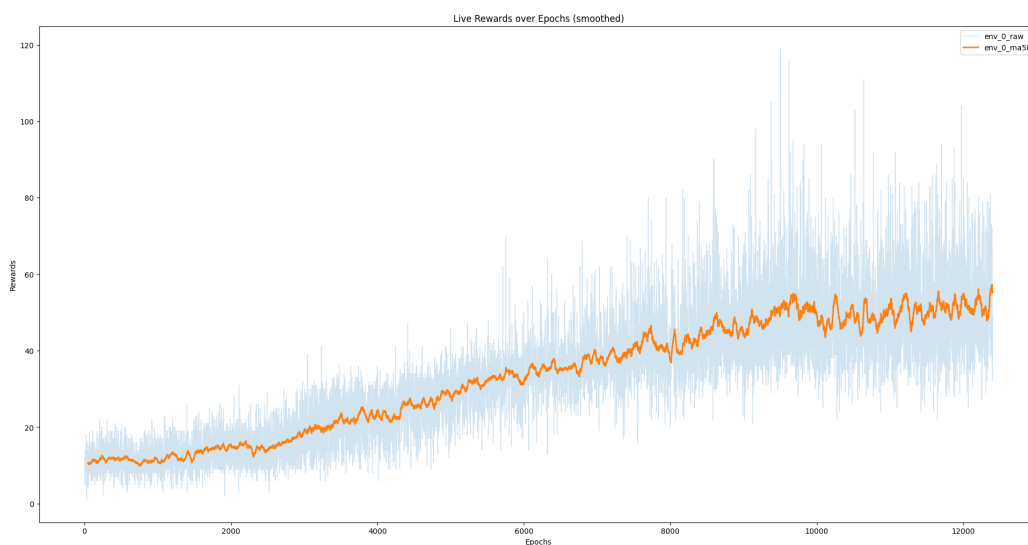


图 4-2 DQN 训练奖励曲线

从曲线可以看出：A2C 在训练初期上升较快；DQN 在前期提升较慢，但中后期达到的平均 reward 更高，整体表现更好。

4.2 训练结果分析

DQN 最终效果超出 A2C，我认为主要有两个原因：

- A2C 每次更新只利用最新一批数据，旧经验不会长期重复使用；DQN 采用经验回放机制，可以反复利用历史高价值样本。
- A2C 的损失包含策略损失、价值损失和熵正则项，受参数设置影响较大。若参数没有完全匹配当前任务，训练效果就可能不理想。

4.3 训练耗时分析

在本机配置下，同样训练 10^7 步，用 DQN 训练耗时约 40 小时；而用 A2C 训练耗时仅约 3 小时。这是因为 A2C 使用多个环境并行采样，同时间能执行多个步骤，训练耗时短；而 DQN 主要按单环境顺序交互，执行步骤效率低，训练耗时长。

5. 实验总结

这是我第一次自己独立完成一个完整的强化学习项目。完成本次实验的过程中，我有很多收获：

- 利用线上学习平台，学习了如何构建并训练一个深度神经网络；
- 通过阅读提出 DQN 和 A3C 算法的论文，对两个算法的思想有了更加深入的理解；
- 通过学习他人实现的算法代码，并借助 AI 辅助编程，在确保理解 AI 生成的代码及其正确性的情况下，完成了算法的构建和训练，提升了自己的编程能力和与 AI 合作的能力。

通过这次实验，我不仅加深了对 DQN 和 A2C 算法原理的理解，还提升了实际编程能力和问题分析能力，并在训练成功的那一刻收获了极大的成就感。

参考文献

- [1] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Playing atari with deep reinforcement learning[J]. arXiv preprint arXiv:1312.5602, 2013.

- [2] MNIH V, BADIA A P, MIRZA M, et al. Asynchronous methods for deep reinforcement learning[C]//International conference on machine learning. 2016: 1928-1937.